

CONFIGSHIELD — DETAILED ARCHITECTURE

Multi-vendor network security compliance auditing architecture — implementation-aligned, with the internal capabilities, execution lifecycle, normalized security model, evidence, risk, remediation and reporting layers described as one system.

1. Runtime Architecture

Layer	Implementation / Responsibility
Web Console	Next.js 14.1.0 + TypeScript. Presents audit creation, progress, findings, remediation plans and reports.
API Gateway	FastAPI 0.109. Receives configuration/audit requests, applies scope and lifecycle rules, exposes audit/compliance/training/remediation APIs.
Execution	Celery 5.3.6 workers with Redis 7. Long-running audit work is executed asynchronously; progress is persisted independently.
Source of Truth	PostgreSQL 15 stores configurations, executions, normalized/compliance results, evidence, findings, remediation plans and audit-trail data.
Engine Layer	Ingestion → validation → detection → parsing → universal model → normalization → compliance/control selection → evidence → findings/risk → remediation → executor → reporting.

2. Twelve-Capability Internal Architecture

#	Capability	Architectural role
01	Ingestion	Accepts raw configuration, validates the incoming artifact and establishes content identity/hash.
02	Validation	Checks structural/input safety and rejects malformed or unsupported audit inputs before evaluation.
03	Detection	Identifies vendor/device/platform using the trained detection layer; confidence is retained.
04	Parsing	Vendor-specific parsers build syntax/context-aware representations from raw configuration.
05	Universal Security Model	Provides the common security state used as the bridge between vendor syntax and control evaluation.
06	Normalization	Maps vendor-specific states into universal paths; unknown or ambiguous mappings can enter review/HITL.
07	Compliance / Control Selection	Selects and evaluates applicable control requirements for supported frameworks, including CIS and NIST.
08	Evidence	Builds traceable evidence from configuration, normalized state and control evaluation.
09	Findings / Risk	Converts failed/reviewed controls into findings and calculates severity/risk information.
10	Remediation	Creates remediation plans, parameterizes fixes, supports approval, script generation and rollback states.
11	Audit Executor	Coordinates the audit lifecycle, execution state, progress, cancellation/recovery and persistence.
12	Reporting	Produces structured audit outputs for dashboard/report consumption.

3. Seven-Step Logical Audit Pipeline

1 Validate — ingestion + validation → 2 Detect — vendor/platform detection → 3 Parse — vendor parser/context tree → 4 Normalize — universal model + normalization → 5 Evaluate — control selection + compliance + evidence → 6 Findings — risk/findings + remediation → 7 Report — reporting, coordinated by the audit executor.

Why 7 steps and 12 capabilities? The seven steps are the user-facing execution stages; the twelve capabilities are the internal implementation building blocks. They are not competing counts and are not intended to be a one-to-one mapping.

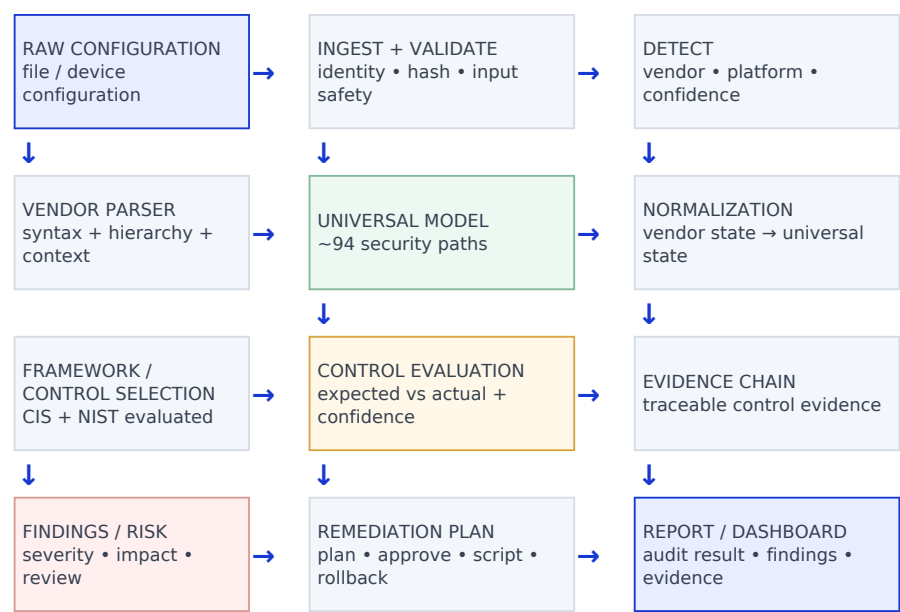
4. Universal Security Model & Normalization

The compliance model is vendor-neutral. Vendor syntax is parsed first, then translated into a common security state before framework evaluation. The current architecture uses approximately 94 universal paths. Representative paths include authentication.aaa.enabled, authentication.password_policy.min_length, management.ssh.version, management.vty.transport, network.cdp.enabled, logging.buffered.level, ntp.authenticated and snmp.v3.privacy.

Context-aware examples: Cisco no cdp run can represent global CDP disablement, while an interface-level no cdp enable has a narrower scope. Likewise, VTY configuration is interpreted with its line-range/context rather than as isolated text.

CONFIGSHIELD — END-TO-END EXECUTION & DATA FLOW

Page 2 visualizes how raw configuration moves through the runtime, 12 internal capabilities and the seven logical audit stages, then becomes evidence-backed findings, remediation and a report.



5. AI / ML Assistance & Human-in-the-Loop

Area	Current role	Safety boundary
Vendor / device detection	TF-IDF + Logistic Regression models assist vendor/device classification.	Confidence is retained; deterministic validation and downstream checks remain authoritative.
Normalization / semantic mapping	TF-IDF + Logistic Regression assists mappings for universal security concepts and semantic interpretation.	Unknown or ambiguous mappings can become REVIEW rather than silently becoming PASS/FAIL.
Risk scoring	A trained risk model supports risk/severity estimation.	Risk output is a decision-support signal, not a replacement for control evidence.
Human-in-the-loop	Admin can inspect AI proposals, confirm/reject them, and feed approved mappings back into the reusable knowledge base.	Human confirmation is the trust boundary for uncertain mappings.

6. Compliance, Evidence & Safety Semantics

The canonical evaluation path is: Configuration → Vendor/Platform → Framework Selection → Control Selection → Normalized Configuration → Control Evaluation → Confidence → Evidence Chain → Control Result → Compliance Result → Findings → Persistence/API/Reports. CIS and NIST are evaluated. STIG references are manual-only, while ISO is treated as a mapping layer. Confidence below 0.70 enters REVIEW. Explicit safety outcomes include unsupported_selection, vendor_mismatch, empty_input and normalization_failed.

7. Remediation & Audit Integrity

Remediation follows a controlled state model: draft → approved → applied → verified, with rolled_back supported. The platform generates operator-executable remediation scripts rather than automatically pushing changes to network devices. This keeps the approval boundary explicit.

Audit integrity uses an append-only, hash-chained audit ledger with sequence, previous-hash and event-hash fields. Remediation events are recorded in the chain, and generated scripts can carry SHA-256 integrity checks. This is tamper-evident audit logging, not blockchain.

8. Execution & Data Invariants

- Raw configuration is validated and hashed at ingestion.
- Deduplication is scoped by device identity and content hash; the same bytes on different devices remain distinct.
- Same-device replay is idempotent; failed retries create new execution records rather than overwriting history.
- Live progress is stored independently from the execution record so progress updates do not lock the main audit lifecycle.
- Cancellation is cooperative; execution can move from cancel_requested to cancelled.
- Company Baseline is a scope projection over stored CIS results, not a second compliance evaluation.

Core architecture principle: vendor syntax is the input language; the Universal Security Model is the common security language; compliance frameworks define expected outcomes; evidence explains why a verdict was produced; remediation turns findings into controlled operator actions.